

Proyecto final de la asignatura Teoría de lenguaje de programación

**UNIVERSIDAD
AUTÓNOMA DE YUCATÁN**

Ingeniería de Software

Teoría de lenguaje de programación

Profesor:

Alejandro Ruiz Pasos

Integrantes del equipo:

Juan Jose Vargas Arvizu

Miguel Angel Gutierrez Lopez

Fecha de entrega: 17/05/2026

ÍNDICE

1. Introducción.....
2. Explicación de Paradigmas de Programación Usados.....
3. Arquitectura del sistema.....
4. Ejemplo de ejecución.....
5. Conclusiones.....

1. Introducción

El diseño de herramientas para la gestión y validación de trayectorias escolares representa un pilar fundamental en la modernización de los sistemas universitarios. Este proyecto consiste en la implementación de un ****Evaluador de Requisitos Académicos (Kárdex)**** concebido como un servicio web basado en el paradigma de la ****lógica declarativa****. La finalidad principal del sistema es procesar consultas lógicas complejas asociadas con la seriación de asignaturas y el progreso crediticio de los alumnos, exponiendo un endpoint estructurado bajo una arquitectura de software de tipo API REST mediante el ecosistema asíncrono de Node.js, Express y la biblioteca Tau Prolog.

2. Explicación de Paradigmas de Programación Usados

El valor técnico del sistema radica en la integración coordinada de múltiples paradigmas de programación, resolviendo una problemática común mediante la división eficiente de responsabilidades: Programación Lógica (Prolog): Empleada como el motor computacional de inferencia. En lugar de modelar los árboles de seriación académica y las restricciones curriculares mediante estructuras condicionales anidadas o iterativas de naturaleza imperativa, este paradigma permite declarar la estructura escolar como un conjunto puro de hechos (materias cursadas, créditos acumulados) y reglas lógicas (requisitos de asignaturas avanzadas). Programación Funcional (JavaScript): Utilizada de manera transversal en el backend web para la manipulación y la sanitización de los datos enviados por los clientes.

El paso de flujos de control mediante funciones puras y la inmutabilidad de los payloads recibidos garantizan una traducción limpia e incorruptible desde las peticiones JSON hacia las consultas lógicas nativas interpretadas por el motor. Programación Asíncrona (Modelo de Node.js): Fundamental para responder de forma concurrente a múltiples clientes. Debido a que la unificación y resolución de reglas en árboles lógicos puede provocar demoras computacionales, la encapsulación del intérprete lógico dentro de ****Promesas**** y su consumo directo mediante estructuras `async/await` evita el bloqueo del hilo de ejecución principal (Event Loop), dotando al servicio de alta disponibilidad y tolerancia a cargas de trabajo concurrentes.

3. Arquitectura del Sistema

El software se compone de un módulo cohesivo e independiente que unifica las capas del servidor web y del procesador lógico a través de cuatro niveles funcionales:

Componente	Responsabilidad Técnica	Tecnología o Elemento Empleado
Servidor HTTP	Publicación del servicio REST, escucha activa en puertos y mapeo de la ruta pública del sistema.	Framework Express, Endpoint de tipo POST (/query)
Capa Asíncrona	Encapsulación no bloqueante del ciclo de vida de las sesiones del intérprete lógico.	Estructuras de control async/await y Promesas de JavaScript
Motor de Inferencia	Carga, instanciación, compilación y búsqueda de unificaciones de la consulta sobre el universo de axiomas.	Biblioteca Tau Prolog embebida de manera nativa en Node.js
Base de Conocimiento	Fichero estático que almacena el modelo de datos de los estudiantes (Kárdex virtual) y las leyes curriculares.	Archivo declarativo con extensión física .pl

4. Ejemplo de ejecución

```

juanito@juanito-surface:~
juanito@juanito-surface:~/Documentos/evaluad | juanito@juanito-surface:~ x

juanito@juanito-surface:~$ curl -X POST http://localhost:3000/query \
  -H "Content-Type: application/json" \
  -d '{"query": "puede_cursar(juan, compiladores)."}'
{"success":true,"result":"true"}
juanito@juanito-surface:~$ curl -X POST http://localhost:3000/query \
  -H "Content-Type: application/json" \
  -d '{"query": "puede_cursar(juan, diseño_sistemas)."}'
{"success":false,"error":"Error de formato en la consulta Prolog: throw(error(syntax_error(unexpected token),[line(1),column(23),found(ño_sistemas).)])"}
juanito@juanito-surface:~$ curl -X POST http://localhost:3000/query \
  -H "Content-Type: application/json" \
  -d '{"query": "aprobada(juan, X)."}'
{"success":true,"result":"X = matematicas_discretas"}
juanito@juanito-surface:~$
```

5. Conclusiones

La culminación de este desarrollo permite demostrar que el paradigma declarativo no es un enfoque aislado de la ingeniería de software actual, sino que puede convivir en perfecta armonía con arquitecturas distribuidas y APIs empresariales modernas. La separación de responsabilidades garantiza que la lógica de negocio (las reglas de la universidad) pueda modificarse en el archivo Prolog de forma inmediata sin alterar una sola línea del servidor HTTP Express. Como ****extensiones futuras**** para el robustecimiento de este sistema, se proponen dos integraciones clave: 1) La migración del archivo estático de Prolog hacia una persistencia dinámica acoplada a bases de datos relacionales, permitiendo cargar el kárdex en tiempo real; y 2) El desarrollo de algoritmos en la capa funcional para retornar explicaciones detalladas (trazas lógicas) en formato JSON cuando una consulta sea rechazada, permitiendo conocer exactamente qué prerequisite o cuántos créditos le faltaron al estudiante para cursar la materia.